

Condensate Partition Coefficient Pipeline

Instruction Manual

Automated 3D segmentation of nuclear biomolecular condensates and measurement of the partition coefficient from confocal Z-stacks

Franco Lab • University of California, Los Angeles

Author: Daniel Chang

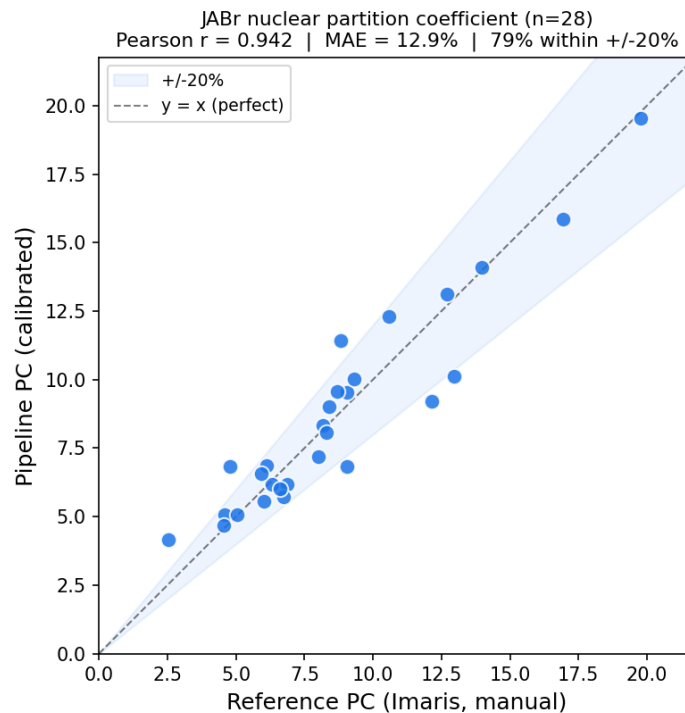
Principal Investigator: _____

Spring 2026

Overview

This pipeline replaces the manual Imaris workflow for measuring the **partition coefficient (PC)** of nuclear biomolecular condensates. Given a two-channel confocal Z-stack, it segments nuclei and condensates in 3D, computes the PC, and reports a calibrated value on the manual reference scale.

Item	Description
Inputs	Two-channel Z-stack: channel 0 = nuclei, channel 1 = condensate.
Outputs	Nuclear & cytoplasmic PC, 3D masks, per-object volumes, summary figure.
Validated on	JABr: Pearson $r = 0.942$, MAE 12.9%, 79% of cells within $\pm 20\%$ ($n = 28$).
Interfaces	Graphical (run_gui.py) or command line (pipeline.py).



JABr validation: calibrated pipeline PC vs manual Imaris reference.

Installation (one time)

1. Install Python 3.10 or 3.11.
2. Install PyTorch for your machine from pytorch.org (CPU or your CUDA version), e.g. CUDA 12.1:

```
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

3. Install the remaining dependencies:

```
pip install -r requirements.txt
```

On first run, Cellpose downloads its model weights automatically (one-time ~100 MB). A GPU is recommended (~10–15 s per cell); CPU works but is slower.

Preparing your image files

The pipeline accepts either one multi-channel TIF (both channels in one file) or two separate single-channel TIFs. **Channel order matters:** channel 0 must be nuclei, channel 1 the condensate. Most axis orderings (ZCYX, CZYX, ...) are auto-detected. If results look wrong, swapped channels are the most common cause.

Running with the graphical interface

Launch with `python run_gui.py`. A window with **Single File** and **Batch** tabs opens.

Step by step (single cell)

1. On the **Single File** tab, choose the input mode and Browse to your data.
2. Choose an Output folder (optional).
3. In **Settings**, set **Construct = JABr**; leave all other settings at their defaults.
4. Click  **Run Pipeline**. Progress streams in the Output Log; the status reads **Done** when finished.
5. Open your output folder for the masks, tables, and results.png.

Live GUI screenshots should be captured on the machine where it runs (Windows: Win+Shift+S) and saved into docs/figures/. The actual numerical output of a real run is shown under “Reading the outputs.”

Understanding the settings

For routine JABr analysis you only need to set **Construct**; the rest have sensible defaults.

Setting	Default	What it does
Construct	(none)	Pick JABr for the validated workflow. Auto-selects the blob_log detector and the JABr calibration.
Condensate detector	auto	auto routes JABr→blob_log, else Cellpose. Normally left on auto.
Top-X% brightest	75	Defines condensed-phase density; trims the dim mask halo to match the manual reference.
Nuclei cell-prob.	-2.0	How aggressively nuclei are detected. -2.0 suits the lab's stain.
blob_log threshold	0.03	Condensate spot sensitivity (used when detector = blob_log). Leave at 0.03 for JABr.
Cellpose cond. model	(blank)	Advanced; a fine-tuned condensate model. Leave blank for JABr.
Disable GPU	off	Tick on a laptop with no NVIDIA GPU.

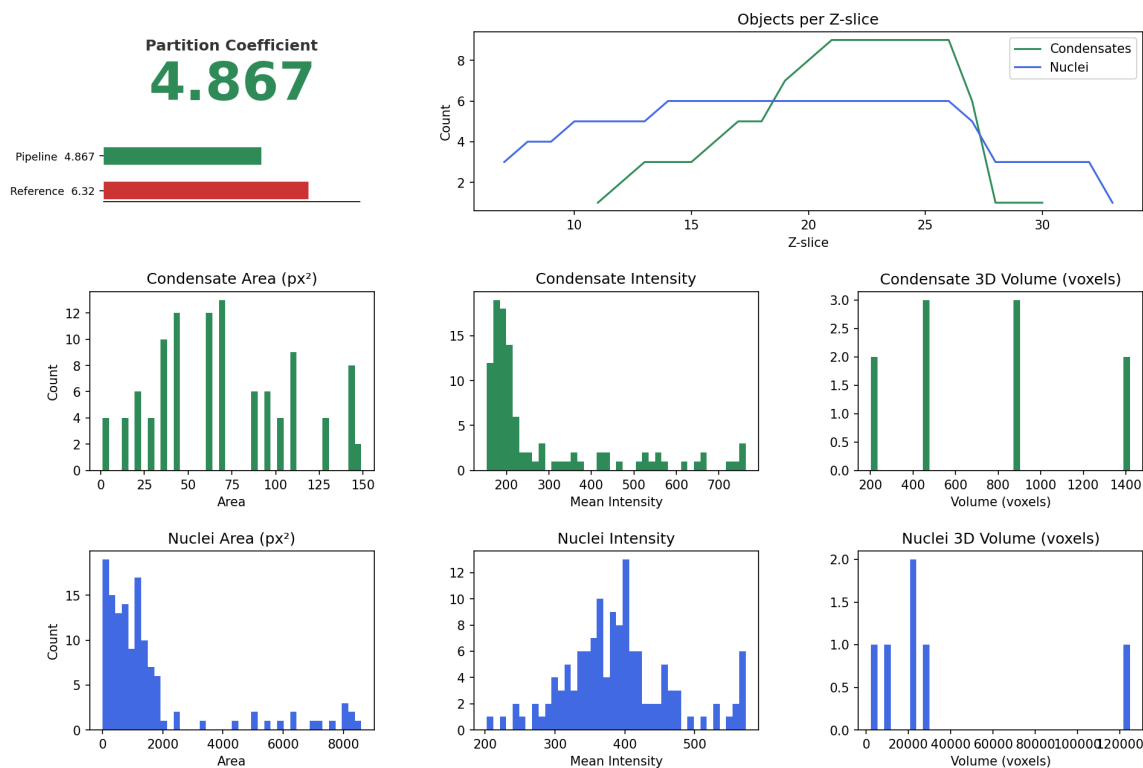
The one rule for routine use: set Construct = JABr and run. Everything else is for experimentation or other constructs.

Reading the outputs

Every run writes summary.csv (headline numbers incl. the calibrated PC), results.png (summary figure), condensate_masks.tif and nuclei_masks.tif (3D labeled masks), and per-object/per-slice CSVs.

For the bundled sample JABr_Sample2_5_3.tif, a real run produces **nuclear PC raw 4.867** → **calibrated 4.813** (manual reference 4.558, +5.6%).

Report the calibrated nuclear PC. The raw value is systematically ~3× the manual scale by construction; calibration standardizes it. **Always glance at condensate_masks.tif** over the raw channel in Fiji/ImageJ — a 10-second check catches the rare misfire.



results.png from the bundled-sample run.

Batch mode and command line

On the **Batch** tab, select a folder of .tif files; every file is processed. Optionally supply the manual Imaris nuclear-PC CSV as a Reference CSV — the pipeline then writes comparison.csv and scatter.png with the correlation, RMSE, and MAE.

Command line:

```
python pipeline.py --roi sample_data/JABr_Sample2_5_3.tif --construct JABr --output my_output
```

Separate channels: --nuc nuclei.tif --cond condensate.tif. Physical volumes in μm^3 : add --voxel-xy 0.065 --voxel-z 0.3. Force CPU: --no-gpu.

Troubleshooting

Symptom	Likely cause / fix
PC wildly off / nuclei empty	Channels swapped: confirm ch0 = nuclei, ch1 = condensate.
"0 condensates detected"	Not JABr, or very dim signal. Lower blob_log threshold (e.g. 0.02); confirm the raw channel has s
Very slow (minutes/cell)	Running on CPU. Install a CUDA PyTorch build; ensure Disable GPU is unchecked.
CUDA out of memory	Tick Disable GPU, or close other GPU programs.
Cellpose download fails	First run needs internet for model weights.
One cell's calibrated PC off	Calibration is population-level; cells vary ($\pm 20\%$ for $\sim 80\%$ of cells). Verify against the mask.

Method summary and scope

The PC is the ratio of condensed-phase to dilute-phase density (Fabrini et al.), both camera-background-subtracted. Condensed density uses the brightest 75% of voxels inside (condensate $\cap$ nucleus); dilute density is the mean of the 50 lowest-intensity $10 \times 10 \times 10$ patches in the nuclear dilute region. Nuclei are segmented with Cellpose 3 (cyto3, 3D), cleaned, and void-filled; condensates are detected with Laplacian-of-Gaussian blob_log and kept when $\geq 50\%$ of the rendered sphere lies inside a nucleus. A per-construct linear (JABr) or isotonic calibration standardizes the automated PC onto the manual scale.

Scope. Validated and production-ready for JABr. The detector is construct-specific; a leave-one-construct-out test confirmed a single detector does not generalize zero-shot (it under-detects on unseen constructs). Extending to other constructs needs per-construct calibration or few-shot retraining. Full rationale and the cross-construct evidence are in docs/METHODS.md.